

Практическое занятие 1.

Разработка Web-приложения баз данных с применением чистой архитектуры

Предметная область — «Бюро переводов». Платформа — .NET 10.0, ASP.NET Core MVC, Entity Framework Core 10, MS SQL Server.

Продолжительность занятия — 90 минут.

Замечание о выборе варианта

Настоящее занятие проводится в трёх независимых вариантах, различающихся применяемой программной архитектурой: вариант 1 — монолитная архитектура, вариант 2 — традиционная многослойная (трёхслойная) архитектура, вариант 3 — чистая архитектура. Обучающийся выполняет один вариант по собственному выбору либо по указанию преподавателя. Варианты являются самостоятельными и не требуют выполнения друг друга: постановка задачи во всех вариантах совпадает, различается исключительно способ организации программного кода.

Цель занятия

Целью занятия является практическое освоение принципов чистой архитектуры: разделения программной системы на доменное ядро, слой сценариев использования, слой инфраструктуры и слой представления, соблюдения правила зависимостей, направленных к доменному ядру, а также приобретение навыков создания доменных сущностей, обладающих поведением, и применения принципа инверсии зависимостей.

Постановка задачи

Требуется разработать Web-приложение, обеспечивающее ведение справочника переводчиков бюро переводов. Приложение должно предоставлять следующие возможности:

- отображение перечня переводчиков с постраничным выводом;
- сортировку перечня по фамилии, имени и отчеству, а также по ставке оплаты;
- фильтрацию перечня по квалификационной категории и по признаку сотрудничества с бюро в настоящее время;
- добавление сведений о новом переводчике с проверкой корректности вводимых данных;
- изменение сведений о существующем переводчике;
- перевод записи о переводчике в архивное состояние.

Сведения о переводчике включают фамилию, имя и отчество, номер телефона, адрес электронной почты, квалификационную категорию, ставку оплаты за единицу объёма работы и признак того, что переводчик сотрудничает с бюро в настоящее время.

Результаты, которые должны быть достигнуты к завершению занятия

- понимать, в каком слое размещается тот или иной вид программного кода: контроллеры относятся к слою представления, сценарии использования — к слою приложения, реализации репозитория — к слою инфраструктуры, правила предметной области — к доменному ядру;
- понимать существо принципа инверсии зависимостей и уметь применять его посредством размещения интерфейсов репозитория в доменном ядре;
- уметь создавать доменные сущности, обладающие поведением, состояние которых изменяется исключительно посредством их методов;

- Недостатком является увеличение количества проектов, файлов и объёма кода преобразования, вследствие чего применение чистой архитектуры при разработке приложений небольшого объёма не всегда оправдано.

Предварительные требования

На рабочем месте должно быть установлено следующее программное обеспечение:

- пакет .NET 10.0 SDK (версия с длительной поддержкой);
- интегрированная среда разработки Visual Studio 2026 с рабочей нагрузкой «ASP.NET и разработка веб-приложений» либо Visual Studio Code с расширением C# Dev Kit;
- система управления базами данных MS SQL Server редакции Developer, Express либо LocalDB;
- средство командной строки `dotnet-ef`.

Проверка установленной версии пакета средств разработки выполняется командой:

```
dotnet --list-sdks
```

Структура решения, получаемая в результате выполнения занятия

```
TranslationBureau.slnx
|
+-- TranslationBureau.Domain                (Class Library, net10.0)
|                                           Ссылка на другие проекты не имеет
|   |
|   +-- Common/
|   |   +-- Entity.cs
|   +-- Entities/
|   |   +-- Translator.cs
|   +-- Exceptions/
|   |   +-- DomainException.cs
|   +-- Interfaces/
|   |   +-- ITranslatorRepository.cs
|
+-- TranslationBureau.Application           (Class Library, net10.0)
|                                           Ссылается на Domain
|   |
|   +-- Common/
|   |   +-- PagedResult.cs
|   +-- DTOs/
|   |   +-- TranslatorDto.cs
|   |   +-- TranslatorInputDto.cs
|   |   +-- TranslatorFilterDto.cs
|   +-- Interfaces/
|   |   +-- ITranslatorService.cs
|   +-- Mapping/
|   |   +-- MappingExtensions.cs
|   +-- Services/
|   |   +-- TranslatorService.cs
|   +-- DependencyInjection.cs
|
+-- TranslationBureau.Infrastructure         (Class Library, net10.0)
|                                           Ссылается на Domain и Application
|   |
|   +-- Persistence/
|   |   +-- ApplicationDbContext.cs
|   |   +-- Configurations/
|   |   |   +-- TranslatorConfiguration.cs
|   |   +-- Repositories/
|   |   |   +-- TranslatorRepository.cs
|   +-- Migrations/
|   +-- DependencyInjection.cs
|
+-- TranslationBureau.Web                   (ASP.NET Core MVC, net10.0)
|                                           Ссылается на Application и Infrastructure
|   |
|   +-- Controllers/
|   |   +-- TranslatorsController.cs
|   +-- ViewModels/
|   |   +-- TranslatorIndexViewModel.cs
|   +-- Views/
|   |   +-- Translators/
|   |   |   +-- Index.cshtml, Create.cshtml, Edit.cshtml, Delete.cshtml
|   +-- wwwroot/
|   +-- appsettings.json
|   +-- Program.cs
```

При недостатке учебного времени приоритет отдаётся шагам 1–12 в объёме операций отображения перечня и добавления записи. Постраничный вывод, сортировка и фильтрация в этом случае выносятся на самостоятельную работу.

Пошаговая инструкция выполнения

Шаг 1. Создание решения, проектов и установление ссылок

1. Откройте терминал в каталоге, предназначенном для размещения решения.
2. Выполните приведённую ниже последовательность команд.

```
dotnet new sln -n TranslationBureau

dotnet new classlib -n TranslationBureau.Domain -f net10.0
dotnet new classlib -n TranslationBureau.Application -f net10.0
dotnet new classlib -n TranslationBureau.Infrastructure -f net10.0
dotnet new mvc -n TranslationBureau.Web -f net10.0

dotnet sln add TranslationBureau.Domain/TranslationBureau.Domain.csproj
dotnet sln add TranslationBureau.Application/TranslationBureau.Application.csproj
dotnet sln add TranslationBureau.Infrastructure/TranslationBureau.Infrastructure.csproj
dotnet sln add TranslationBureau.Web/TranslationBureau.Web.csproj
```

3. Установите ссылки между проектами.

```
dotnet add TranslationBureau.Application reference TranslationBureau.Domain
dotnet add TranslationBureau.Infrastructure reference TranslationBureau.Domain
dotnet add TranslationBureau.Infrastructure reference TranslationBureau.Application
dotnet add TranslationBureau.Web reference TranslationBureau.Application
dotnet add TranslationBureau.Web reference TranslationBureau.Infrastructure
```

Следует особо подчеркнуть, что проект `TranslationBureau.Domain` не содержит ни одной ссылки на другие проекты решения. Нарушение указанного правила означает отказ от принципов чистой архитектуры.

4. Удалите из проектов библиотек классов автоматически созданные файлы `Class1.cs`.

```
del TranslationBureau.Domain\Class1.cs
del TranslationBureau.Application\Class1.cs
del TranslationBureau.Infrastructure\Class1.cs
```

5. Убедитесь в успешности сборки решения командой `dotnet build`.

Начиная с версии .NET 10.0 команда `dotnet new sln` по умолчанию создаёт файл решения формата `.slnx` — XML-файл, обладающий существенно более компактной структурой по сравнению с файлом формата `.sln`. Формат задаётся параметром `--format`: команда `dotnet new sln -n TranslationBureau --format sln` создаёт файл прежнего формата. Преобразование ранее созданного файла формата `.sln` выполняется командой `dotnet sln migrate`. После добавления проектов содержимое файла `TranslationBureau.slnx` является следующим.

```
<Solution>
  <Project Path="TranslationBureau.Application/TranslationBureau.Application.csproj" />
  <Project Path="TranslationBureau.Domain/TranslationBureau.Domain.csproj" />
  <Project Path="TranslationBureau.Infrastructure/TranslationBureau.Infrastructure.csproj" />
  <Project Path="TranslationBureau.Web/TranslationBureau.Web.csproj" />
</Solution>
```

Шаг 2. Установка пакетов NuGet

Пакеты устанавливаются раздельно по слоям в соответствии с назначением каждого слоя.

```
:: Доменный слой пакетов не требует: он не зависит ни от технологии
:: доступа к данным, ни от инфраструктуры приложения

:: Слой приложения
dotnet add TranslationBureau.Application package Microsoft.Extensions.DependencyInjection.Abstractions
dotnet add TranslationBureau.Application package Microsoft.EntityFrameworkCore

:: Слой инфраструктуры
dotnet add TranslationBureau.Infrastructure package Microsoft.EntityFrameworkCore.SqlServer
dotnet add TranslationBureau.Infrastructure package Microsoft.EntityFrameworkCore.Design
dotnet add TranslationBureau.Infrastructure package Microsoft.Extensions.Configuration.Abstractions

:: Слой представления
dotnet add TranslationBureau.Web package Microsoft.EntityFrameworkCore.Design
dotnet add TranslationBureau.Web package Microsoft.AspNetCore.Diagnostics.EntityFrameworkCore
```

Установка пакета `Microsoft.EntityFrameworkCore` в слой приложения требует пояснения. В указанном пакете объявлены методы асинхронного получения результата — `ToListAsync`, `CountAsync` и другие, — применяемые к выражениям типа `IQueryable<T>`, которые возвращают репозитории. Провайдер конкретной системы управления базами данных в этот слой не устанавливается: выбор провайдера остаётся за слоем инфраструктуры, вследствие чего независимость слоя приложения от системы управления базами данных сохраняется.

Номер версии в приведённых командах не указывается: средство `dotnet add package` устанавливает последнюю выпущенную версию пакета из ветви 10.0.x. Явное закрепление версии `10.0.0` не рекомендуется, поскольку указанная версия влечёт установку зависимого пакета `System.Security.Cryptography.Xml` версии 9.0.0, для которого опубликованы сведения об уязвимостях, вследствие чего сборка проекта сопровождается предупреждениями `NU1903`.

При отсутствии средства командной строки Entity Framework Core оно устанавливается командой `dotnet tool install --global dotnet-ef`.

Шаг 3. Создание доменного ядра

- Создайте в проекте `TranslationBureau.Domain` папку `Exceptions` и добавьте в неё файл `DomainException.cs`.

Файл `TranslationBureau.Domain/Exceptions/DomainException.cs`:

```
namespace TranslationBureau.Domain.Exceptions;

/// <summary>Исключение, возбуждаемое при нарушении правила предметной области.</summary>
public class DomainException : Exception
{
    public DomainException(string message) : base(message)
    {
    }
}
```

- Создайте папку `Common` и добавьте в неё файл `Entity.cs`.

Файл `TranslationBureau.Domain/Common/Entity.cs`:

```
namespace TranslationBureau.Domain.Common;

/// <summary>Базовый класс доменной сущности, обладающей идентификатором.</summary>
public abstract class Entity
{
    public int Id { get; protected set; }
```

```

public override bool Equals(object? obj)
{
    if (obj is not Entity other || GetType() != other.GetType())
    {
        return false;
    }

    // Сущности, не сохранённые в хранилище, сравниваются по ссылке
    if (Id == 0 || other.Id == 0)
    {
        return ReferenceEquals(this, other);
    }

    return Id == other.Id;
}

public override int GetHashCode() => GetHashCode.Combine(GetType(), Id);
}

```

8. Создайте папку **Entities** и добавьте в неё файл **Translator.cs**.

Файл **TranslationBureau.Domain/Entities/Translator.cs**:

```

using TranslationBureau.Domain.Common;
using TranslationBureau.Domain.Exceptions;

namespace TranslationBureau.Domain.Entities;

/// <summary>Переводчик бюро переводов.</summary>
public class Translator : Entity
{
    /// <summary>Конструктор без параметров требуется Entity Framework Core.</summary>
    private Translator()
    {
    }

    private Translator(string fullName, string? phone, string? email,
        string? category, decimal ratePerUnit)
    {
        FullName = fullName;
        Phone = phone;
        Email = email;
        Category = category;
        RatePerUnit = ratePerUnit;
        IsActive = true;
    }

    public string FullName { get; private set; } = string.Empty;
    public string? Phone { get; private set; }
    public string? Email { get; private set; }
    public string? Category { get; private set; }
    public decimal RatePerUnit { get; private set; }
    public bool IsActive { get; private set; }

    /// <summary>Фабричный метод создания переводчика с проверкой правил
    /// предметной области.</summary>
    public static Translator Create(string fullName, string? phone, string? email,
        string? category, decimal ratePerUnit)
    {
        Validate(fullName, ratePerUnit);

        return new Translator(fullName.Trim(), phone, email, category, ratePerUnit);
    }

    /// <summary>Изменяет реквизиты переводчика.</summary>
    public void Update(string fullName, string? phone, string? email,
        string? category, decimal ratePerUnit)
    {
    }
}

```

```

{
    Validate(fullName, ratePerUnit);

    FullName = fullName.Trim();
    Phone = phone;
    Email = email;
    Category = category;
    RatePerUnit = ratePerUnit;
}

/// <summary>Переводит запись в архивное состояние вместо физического удаления.</summary>
public void Deactivate() => IsActive = false;

public void Activate() => IsActive = true;

private static void Validate(string fullName, decimal ratePerUnit)
{
    if (string.IsNullOrEmpty(fullName))
    {
        throw new DomainException("ФИО переводчика не может быть пустым.");
    }

    if (ratePerUnit <= 0)
    {
        throw new DomainException("Ставка переводчика должна быть положительной.");
    }
}
}

```

Обратите внимание на следующие существенные обстоятельства. Все свойства сущности объявлены с закрытыми методами установки значений, вследствие чего изменение состояния сущности извне невозможно. Создание сущности выполняется исключительно фабричным методом **Create**, изменение — методом **Update**; оба метода проверяют правила предметной области. Такое построение исключает существование сущности в недопустимом состоянии. Атрибуты проверки вводимых данных в доменной сущности не применяются, поскольку они относятся к пространству имён, предназначенному для слоя представления.

Шаг 4. Объявление интерфейса репозитория в доменном ядре

Создайте в проекте **TranslationBureau.Domain** папку **Interfaces** и добавьте в неё файл **ITranslatorRepository.cs**.

Файл **TranslationBureau.Domain/Interfaces/ITranslatorRepository.cs**:

```

using TranslationBureau.Domain.Entities;

namespace TranslationBureau.Domain.Interfaces;

public interface ITranslatorRepository
{
    /// <summary>Возвращает запрос, допускающий последующее уточнение.</summary>
    IQueryable<Translator> Query();

    Task<Translator?> GetByIdAsync(int id, CancellationToken cancellationToken = default);

    Task AddAsync(Translator translator, CancellationToken cancellationToken = default);

    Task<int> SaveChangesAsync(CancellationToken cancellationToken = default);
}

```

Размещение интерфейса репозитория именно в доменном ядре составляет существо принципа инверсии зависимостей. Доменное ядро объявляет требуемый ему контракт, не располагая при этом никакими сведениями о способе его реализации. Слой инфраструктуры, зависящий от доменного ядра,

реализует указанный контракт средствами Entity Framework Core. Вследствие этого зависимость направлена от инфраструктуры к ядру, а не наоборот.

Шаг 5. Создание вспомогательных типов и объектов передачи данных

9. Создайте в проекте `TranslationBureau.Application` папку `Common` и добавьте в неё файл `PagedResult.cs`.

Файл `TranslationBureau.Application/Common/PagedResult.cs`:

```
namespace TranslationBureau.Application.Common;

/// <summary>Результат постраничной выборки элементов произвольного типа.</summary>
public class PagedResult<T>
{
    public IReadOnlyList<T> Items { get; init; } = [];
    public int PageIndex { get; init; }
    public int PageSize { get; init; }
    public int TotalCount { get; init; }

    public int TotalPages => (int)Math.Ceiling(TotalCount / (double)PageSize);
    public bool HasPreviousPage => PageIndex > 1;
    public bool HasNextPage => PageIndex < TotalPages;
}
```

10. Создайте папку `DTOs` и добавьте в неё файл `TranslatorDto.cs`.

Файл `TranslationBureau.Application/DTOs/TranslatorDto.cs`:

```
namespace TranslationBureau.Application.DTOs;

/// <summary>Объект передачи сведений о переводчике в слой представления.</summary>
public class TranslatorDto
{
    public int Id { get; init; }
    public string FullName { get; init; } = string.Empty;
    public string? Phone { get; init; }
    public string? Email { get; init; }
    public string? Category { get; init; }
    public decimal RatePerUnit { get; init; }
    public bool IsActive { get; init; }
}
```

11. Добавьте файл `TranslatorInputDto.cs`.

Файл `TranslationBureau.Application/DTOs/TranslatorInputDto.cs`:

```
using System.ComponentModel.DataAnnotations;

namespace TranslationBureau.Application.DTOs;

/// <summary>Объект передачи данных, вводимых пользователем.</summary>
public class TranslatorInputDto
{
    public int Id { get; set; }

    [Display(Name = "ФИО")]
    [Required(ErrorMessage = "Укажите ФИО переводчика")]
    [StringLength(150, ErrorMessage = "Длина ФИО не должна превышать 150 символов")]
    public string FullName { get; set; } = string.Empty;

    [Display(Name = "Телефон")]
    [Phone(ErrorMessage = "Некорректный номер телефона")]
    [StringLength(20)]
    public string? Phone { get; set; }

    [Display(Name = "Электронная почта")]
    [EmailAddress(ErrorMessage = "Некорректный адрес электронной почты")]
}
```

```

[StringLength(100)]
public string? Email { get; set; }

[Display(Name = "Категория")]
[StringLength(50)]
public string? Category { get; set; }

[Display(Name = "Ставка за единицу, руб.")]
[Range(0.01, 100000, ErrorMessage = "Ставка должна быть положительной")]
public decimal RatePerUnit { get; set; }
}

```

12. Добавьте файл `TranslatorFilterDto.cs`.

Файл `TranslationBureau.Application/DTOs/TranslatorFilterDto.cs`:

```

namespace TranslationBureau.Application.DTOs;

/// <summary>Условия отбора и упорядочения перечня переводчиков.</summary>
public class TranslatorFilterDto
{
    public string? Category { get; set; }
    public bool? IsActive { get; set; }
    public string? SortOrder { get; set; }
    public int Page { get; set; } = 1;
    public int PageSize { get; set; } = 5;
}

```

Шаг 6. Создание методов преобразования

Создайте в проекте `TranslationBureau.Application` папку `Mapping` и добавьте в неё файл `MappingExtensions.cs`.

Файл `TranslationBureau.Application/Mapping/MappingExtensions.cs`:

```

using TranslationBureau.Application.DTOs;
using TranslationBureau.Domain.Entities;

namespace TranslationBureau.Application.Mapping;

/// <summary>Методы преобразования доменных сущностей в объекты передачи данных.</summary>
public static class MappingExtensions
{
    public static TranslatorDto ToDto(this Translator translator) => new()
    {
        Id = translator.Id,
        FullName = translator.FullName,
        Phone = translator.Phone,
        Email = translator.Email,
        Category = translator.Category,
        RatePerUnit = translator.RatePerUnit,
        IsActive = translator.IsActive
    };

    public static TranslatorInputDto ToInputDto(this Translator translator) => new()
    {
        Id = translator.Id,
        FullName = translator.FullName,
        Phone = translator.Phone,
        Email = translator.Email,
        Category = translator.Category,
        RatePerUnit = translator.RatePerUnit
    };
}

```

Преобразование выполняется исключительно в направлении от доменной сущности к объекту передачи данных. Обратное преобразование не реализуется: создание и изменение сущности выполняются вызовом её методов **Create** и **Update**, что исключает обход правил предметной области.

Шаг 7. Объявление и реализация службы слоя приложения

13. Создайте в проекте **TranslationBureau.Application** папку **Interfaces** и добавьте в неё файл **ITranslatorService.cs**.

Файл **TranslationBureau.Application/Interfaces/ITranslatorService.cs**:

```
using TranslationBureau.Application.Common;
using TranslationBureau.Application.DTOS;

namespace TranslationBureau.Application.Interfaces;

public interface ITranslatorService
{
    Task<PagedResult<TranslatorDto>> GetPagedAsync(TranslatorFilterDto filter,
        CancellationToken cancellationToken = default);

    Task<IReadOnlyList<string>> GetCategoriesAsync(
        CancellationToken cancellationToken = default);

    Task<TranslatorDto?> GetByIdAsync(int id, CancellationToken cancellationToken = default);

    Task<TranslatorInputDto?> GetForEditAsync(int id,
        CancellationToken cancellationToken = default);

    Task<int> CreateAsync(TranslatorInputDto dto,
        CancellationToken cancellationToken = default);

    Task UpdateAsync(TranslatorInputDto dto, CancellationToken cancellationToken = default);

    Task DeactivateAsync(int id, CancellationToken cancellationToken = default);
}
```

14. Создайте папку **Services** и добавьте в неё файл **TranslatorService.cs**.

Файл **TranslationBureau.Application/Services/TranslatorService.cs**:

```
using Microsoft.EntityFrameworkCore;
using TranslationBureau.Application.Common;
using TranslationBureau.Application.DTOS;
using TranslationBureau.Application.Interfaces;
using TranslationBureau.Application.Mapping;
using TranslationBureau.Domain.Entities;
using TranslationBureau.Domain.Exceptions;
using TranslationBureau.Domain.Interfaces;

namespace TranslationBureau.Application.Services;

public class TranslatorService : ITranslatorService
{
    private readonly ITranslatorRepository _repository;

    public TranslatorService(ITranslatorRepository repository)
        => _repository = repository;

    public async Task<PagedResult<TranslatorDto>> GetPagedAsync(
        TranslatorFilterDto filter, CancellationToken cancellationToken = default)
    {
        var query = _repository.Query();

        // Отбор записей по заданным условиям
        if (!string.IsNullOrEmpty(filter.Category))

```

```

    {
        query = query.Where(t => t.Category == filter.Category);
    }

    if (filter.IsActive.HasValue)
    {
        query = query.Where(t => t.IsActive == filter.IsActive.Value);
    }

    // Упорядочение записей
    query = filter.SortOrder switch
    {
        "name_desc" => query.OrderByDescending(t => t.FullName),
        "rate" => query.OrderBy(t => t.RatePerUnit),
        "rate_desc" => query.OrderByDescending(t => t.RatePerUnit),
        _ => query.OrderBy(t => t.FullName)
    };

    var totalCount = await query.CountAsync(cancellationTokens);

    var items = await query
        .Skip((filter.Page - 1) * filter.PageSize)
        .Take(filter.PageSize)
        .ToListAsync(cancellationTokens);

    return new PagedResult<TranslatorDto>
    {
        Items = items.Select(t => t.ToDto()).ToList(),
        PageIndex = filter.Page,
        PageSize = filter.PageSize,
        TotalCount = totalCount
    };
}

public async Task<IReadOnlyList<string>> GetCategoriesAsync(
    CancellationToken cancellationToken = default)
    => await _repository.Query()
        .Where(t => t.Category != null)
        .Select(t => t.Category!)
        .Distinct()
        .OrderBy(c => c)
        .ToListAsync(cancellationTokens);

public async Task<TranslatorDto?> GetByIdAsync(int id,
    CancellationToken cancellationToken = default)
{
    var translator = await _repository.GetByIdAsync(id, cancellationToken);
    return translator?.ToDto();
}

public async Task<TranslatorInputDto?> GetForEditAsync(int id,
    CancellationToken cancellationToken = default)
{
    var translator = await _repository.GetByIdAsync(id, cancellationToken);
    return translator?.ToInputDto();
}

public async Task<int> CreateAsync(TranslatorInputDto dto,
    CancellationToken cancellationToken = default)
{
    // Правила предметной области проверяются доменной сущностью
    var translator = Translator.Create(dto.FullName, dto.Phone,
        dto.Email, dto.Category, dto.RatePerUnit);

    await _repository.AddAsync(translator, cancellationToken);
    await _repository.SaveChangesAsync(cancellationTokens);

    return translator.Id;
}

```

```

public async Task UpdateAsync(TranslatorInputDto dto,
    CancellationToken cancellationToken = default)
{
    var translator = await _repository.GetByIdAsync(dto.Id, cancellationToken)
        ?? throw new DomainException("Переводчик не найден.");

    translator.Update(dto.FullName, dto.Phone, dto.Email,
        dto.Category, dto.RatePerUnit);

    await _repository.SaveChangesAsync(cancellationToken);
}

public async Task DeactivateAsync(int id, CancellationToken cancellationToken = default)
{
    var translator = await _repository.GetByIdAsync(id, cancellationToken)
        ?? throw new DomainException("Переводчик не найден.");

    // Вместо физического удаления запись переводится в архивное состояние,
    // поскольку на неё могут ссылаться ранее выполненные заказы
    translator.Deactivate();

    await _repository.SaveChangesAsync(cancellationToken);
}
}

```

Обратите внимание на то, что служба слоя приложения не содержит проверок правил предметной области. Указанные проверки размещены в доменной сущности, что обеспечивает их соблюдение при любом способе создания и изменения сущности. Задачей службы является организация последовательности действий: получение сущности из хранилища, вызов её метода и сохранение изменений.

15. Добавьте в корень проекта `TranslationBureau.Application` файл `DependencyInjection.cs`.

Файл `TranslationBureau.Application/DependencyInjection.cs`:

```

using Microsoft.Extensions.DependencyInjection;
using TranslationBureau.Application.Interfaces;
using TranslationBureau.Application.Services;

namespace TranslationBureau.Application;

public static class DependencyInjection
{
    /// <summary>Регистрирует службы слоя приложения.</summary>
    public static IServiceCollection AddApplication(this IServiceCollection services)
    {
        services.AddScoped<ITranslatorService, TranslatorService>();
        return services;
    }
}

```

Шаг 8. Создание слоя инфраструктуры

16. Создайте в проекте `TranslationBureau.Infrastructure` папку `Persistence`, а внутри неё — папку `Configurations`.

17. Добавьте в папку `Configurations` файл `TranslatorConfiguration.cs`.

Файл `TranslationBureau.Infrastructure/Persistence/Configurations/TranslatorConfiguration.cs`:

```

using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using TranslationBureau.Domain.Entities;

```

```

namespace TranslationBureau.Infrastructure.Persistence.Configurations;

/// <summary>Конфигурация отображения сущности «Переводчик» на таблицу.</summary>
public class TranslatorConfiguration : IEntityTypeConfiguration<Translator>
{
    public void Configure(EntityTypeBuilder<Translator> builder)
    {
        builder.ToTable("Translators");
        builder.HasKey(t => t.Id);

        builder.Property(t => t.FullName)
            .IsRequired()
            .HasMaxLength(150);

        builder.Property(t => t.Phone).HasMaxLength(20);
        builder.Property(t => t.Email).HasMaxLength(100);
        builder.Property(t => t.Category).HasMaxLength(50);

        // Явное указание типа столбца исключает предупреждение
        // о возможной потере точности при отображении типа decimal
        builder.Property(t => t.RatePerUnit).HasColumnType("decimal(10,2)");

        builder.HasIndex(t => t.FullName);
    }
}

```

Настройка отображения сущностей на таблицы выполняется в слое инфраструктуры именно потому, что доменное ядро не должно располагать сведениями о структуре базы данных. Entity Framework Core обращается к закрытым методам установки значений свойств средствами отражения, вследствие чего инкапсуляция доменной сущности не препятствует её сохранению.

18. Добавьте в папку **Persistence** файл **ApplicationDbContext.cs**.

Файл **TranslationBureau.Infrastructure/Persistence/ApplicationDbContext.cs**:

```

using Microsoft.EntityFrameworkCore;
using TranslationBureau.Domain.Entities;

namespace TranslationBureau.Infrastructure.Persistence;

/// <summary>Контекст базы данных бюро переводов.</summary>
public class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options)
    {
    }

    public DbSet<Translator> Translators => Set<Translator>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Применение всех классов конфигурации, объявленных в данной сборке
        modelBuilder.ApplyConfigurationsFromAssembly(
            typeof(ApplicationDbContext).Assembly);

        base.OnModelCreating(modelBuilder);
    }
}

```

19. Создайте в папке **Persistence** папку **Repositories** и добавьте в неё файл **TranslatorRepository.cs**.

Файл **TranslationBureau.Infrastructure/Persistence/Repositories/TranslatorRepository.cs**:

```

using Microsoft.EntityFrameworkCore;
using TranslationBureau.Domain.Entities;

```

```

using TranslationBureau.Domain.Interfaces;

namespace TranslationBureau.Infrastructure.Persistence.Repositories;

public class TranslatorRepository : ITranslatorRepository
{
    private readonly ApplicationDbContext _context;

    public TranslatorRepository(ApplicationDbContext context)
        => _context = context;

    public IQueryable<Translator> Query()
        => _context.Translators.AsNoTracking();

    public async Task<Translator?> GetByIdAsync(int id,
        CancellationToken cancellationToken = default)
        => await _context.Translators
            .FirstOrDefaultAsync(t => t.Id == id, cancellationToken);

    public async Task AddAsync(Translator translator,
        CancellationToken cancellationToken = default)
        => await _context.Translators.AddAsync(translator, cancellationToken);

    public Task<int> SaveChangesAsync(CancellationToken cancellationToken = default)
        => _context.SaveChangesAsync(cancellationToken);
}

```

Шаг 9. Регистрация служб слоя инфраструктуры

Добавьте в корень проекта `TranslationBureau.Infrastructure` файл `DependencyInjection.cs`.

Файл `TranslationBureau.Infrastructure/DependencyInjection.cs`:

```

using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using TranslationBureau.Domain.Interfaces;
using TranslationBureau.Infrastructure.Persistence;
using TranslationBureau.Infrastructure.Persistence.Repositories;

namespace TranslationBureau.Infrastructure;

public static class DependencyInjection
{
    /// <summary>Регистрирует контекст базы данных и реализации репозитория.</summary>
    public static IServiceCollection AddInfrastructure(this IServiceCollection services,
        IConfiguration configuration)
    {
        var connectionString = configuration.GetConnectionString("DefaultConnection")
            ?? throw new InvalidOperationException(
                "Строка подключения «DefaultConnection» не найдена.");

        services.AddDbContext<ApplicationDbContext>(options =>
            options.UseSqlServer(connectionString));

        services.AddScoped<ITranslatorRepository, TranslatorRepository>();

        return services;
    }
}

```

Именно данный файл содержит единственное во всём решении упоминание применяемой системы управления базами данных. Замена MS SQL Server на иную систему требует изменения исключительно вызова метода `UseSqlServer` и установленного пакета провайдера.

Шаг 10. Настройка слоя представления

20. Откройте файл `TranslationBureau.Web/appsettings.json` и добавьте раздел `ConnectionStrings`.

```
{
  "ConnectionStrings": {
    "DefaultConnection":
      "Server=(localdb)\\MSSQLLocalDB;Database=TranslationBureauCleanDB;Trusted_Connection=True;MultipleActiveResultSets=true;TrustServerCertificate=True"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

21. Приведите файл `TranslationBureau.Web/Program.cs` к следующему виду.

```
using TranslationBureau.Application;
using TranslationBureau.Infrastructure;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

// Точка сборки приложения: абстракции связываются с конкретными реализациями
builder.Services.AddApplication();
builder.Services.AddInfrastructure(builder.Configuration);

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Translators}/{action=Index}/{id?}");

app.Run();
```

Файл `Program.cs` является единственным местом решения, в котором проект слоя представления обращается к слою инфраструктуры. Именно поэтому ссылка на проект `TranslationBureau.Infrastructure` не нарушает принципов чистой архитектуры: она требуется исключительно в точке сборки.

22. Создайте в проекте `TranslationBureau.Web` папку `ViewModels` и добавьте в неё файл `TranslatorIndexViewModel.cs`.

Файл `TranslationBureau.Web/ViewModels/TranslatorIndexViewModel.cs`:

```
using TranslationBureau.Application.Common;
using TranslationBureau.Application.DTOS;

namespace TranslationBureau.Web.ViewModels;

/// <summary>Модель представления перечня переводчиков.</summary>
```

```

public class TranslatorIndexViewModel
{
    public PagedResult<TranslatorDto> Translators { get; set; } = new();
    public TranslatorFilterDto Filter { get; set; } = new();
    public IReadOnlyList<string> Categories { get; set; } = [];
}

```

Шаг 11. Создание контроллера

Добавьте в папку `TranslationBureau.Web/Controllers` файл `TranslatorsController.cs`.

Файл `TranslationBureau.Web/Controllers/TranslatorsController.cs`:

```

using Microsoft.AspNetCore.Mvc;
using TranslationBureau.Application.DTOS;
using TranslationBureau.Application.Interfaces;
using TranslationBureau.Domain.Exceptions;
using TranslationBureau.Web.ViewModels;

namespace TranslationBureau.Web.Controllers;

public class TranslatorsController : Controller
{
    private readonly ITranslatorService _service;

    public TranslatorsController(ITranslatorService service)
        => _service = service;

    public async Task<IActionResult> Index([FromQuery] TranslatorFilterDto filter,
        CancellationToken cancellationToken)
    {
        var model = new TranslatorIndexViewModel
        {
            Translators = await _service.GetPagedAsync(filter, cancellationToken),
            Filter = filter,
            Categories = await _service.GetCategoriesAsync(cancellationToken)
        };

        return View(model);
    }

    public IActionResult Create() => View(new TranslatorInputDto());

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Create(TranslatorInputDto model,
        CancellationToken cancellationToken)
    {
        if (!ModelState.IsValid)
        {
            return View(model);
        }

        try
        {
            await _service.CreateAsync(model, cancellationToken);
            return RedirectToAction(nameof(Index));
        }
        catch (DomainException exception)
        {
            ModelState.AddModelError(string.Empty, exception.Message);
            return View(model);
        }
    }

    public async Task<IActionResult> Edit(int id, CancellationToken cancellationToken)
    {
        var model = await _service.GetForEditAsync(id, cancellationToken);
    }
}

```

```

        return model is null ? NotFound() : View(model);
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Edit(TranslatorInputDto model,
        CancellationToken cancellationToken)
    {
        if (!ModelState.IsValid)
        {
            return View(model);
        }

        try
        {
            await _service.UpdateAsync(model, cancellationToken);
            return RedirectToAction(nameof(Index));
        }
        catch (DomainException exception)
        {
            ModelState.AddModelError(string.Empty, exception.Message);
            return View(model);
        }
    }

    public async Task<IActionResult> Delete(int id, CancellationToken cancellationToken)
    {
        var model = await _service.GetByIdAsync(id, cancellationToken);
        return model is null ? NotFound() : View(model);
    }

    [HttpPost]
    [ActionName("Delete")]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> DeleteConfirmed(int id,
        CancellationToken cancellationToken)
    {
        await _service.DeactivateAsync(id, cancellationToken);
        return RedirectToAction(nameof(Index));
    }
}

```

Контроллер не содержит ни одного обращения к Entity Framework Core и не располагает сведениями о применяемой системе управления базами данных. Единственной его зависимостью является интерфейс **ITranslatorService**.

Шаг 12. Создание представлений

23. Создайте папку **TranslationBureau.Web/Views/Translators**.

24. Добавьте в неё файл **Index.cshtml**.

Файл **Views/Translators/Index.cshtml**:

```

@using Microsoft.AspNetCore.Mvc.Rendering
@model TranslationBureau.Web.ViewModels.TranslatorIndexViewModel
@{
    ViewData["Title"] = "Переводчики";
    var filter = Model.Filter;
}

<h2>Переводчики</h2>

<form asp-action="Index" method="get" class="row g-2 mb-3">
    <div class="col-auto">
        <select name="category" class="form-select"
            asp-items="@((new SelectList(Model.Categories, filter.Category)))">
            <option value="">Все категории</option>
        </select>
    </div>

```

```

</div>
<div class="col-auto">
    <select name="isActive" class="form-select">
        <option value="">Все переводчики</option>
        <option value="true" selected="@filter.IsActive == true">Работающие</option>
        <option value="false" selected="@filter.IsActive == false">Архивные</option>
    </select>
</div>
<div class="col-auto">
    <button type="submit" class="btn btn-outline-primary">Применить</button>
    <a asp-action="Index" class="btn btn-outline-secondary">Сбросить</a>
</div>
<div class="col-auto ms-auto">
    <a asp-action="Create" class="btn btn-primary">Добавить переводчика</a>
</div>
</form>

<table class="table table-striped table-bordered">
    <thead class="table-light">
        <tr>
            <th>
                <a asp-action="Index"
                    asp-route-sortOrder="@filter.SortOrder == "name" ? "name_desc" : "name")"
                    asp-route-category="@filter.Category"
                    asp-route-isActive="@filter.IsActive">ФИО</a>
            </th>
            <th>Телефон</th>
            <th>Электронная почта</th>
            <th>Категория</th>
            <th class="text-end">
                <a asp-action="Index"
                    asp-route-sortOrder="@filter.SortOrder == "rate" ? "rate_desc" : "rate")"
                    asp-route-category="@filter.Category"
                    asp-route-isActive="@filter.IsActive">Ставка, руб.</a>
            </th>
            <th>Работает</th>
            <th></th>
        </tr>
    </thead>
    <tbody>
        @foreach (var item in Model.Translators.Items)
        {
            <tr>
                <td>@item.FullName</td>
                <td>@item.Phone</td>
                <td>@item.Email</td>
                <td>@item.Category</td>
                <td class="text-end">@item.RatePerUnit.ToString("N2")</td>
                <td>@(item.IsActive ? "да" : "нет")</td>
                <td>
                    <a asp-action="Edit" asp-route-id="@item.Id">Изменить</a> |
                    <a asp-action="Delete" asp-route-id="@item.Id">Удалить</a>
                </td>
            </tr>
        }
    </tbody>
</table>

<nav>
    <ul class="pagination">
        @for (var i = 1; i <= Model.Translators.TotalPages; i++)
        {
            <li class="page-item @(i == Model.Translators.PageIndex ? "active" : "")">
                <a class="page-link" asp-action="Index" asp-route-page="@i"
                    asp-route-category="@filter.Category"
                    asp-route-isActive="@filter.IsActive"
                    asp-route-sortOrder="@filter.SortOrder">@i</a>
            </li>
        }
    </ul>
</nav>

```

```
<p>Всего записей: @Model.Translators.TotalCount</p>
```

25. Добавьте файл `Create.cshtml`.

Файл `Views/Translators/Create.cshtml`:

```
@model TranslationBureau.Application.DTOs.TranslatorInputDto
@{
    ViewData["Title"] = "Добавление переводчика";
}

<h2>Добавление переводчика</h2>

<form asp-action="Create" method="post" class="col-md-6">
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>

    <div class="mb-3">
        <label asp-for="FullName" class="form-label"></label>
        <input asp-for="FullName" class="form-control" />
        <span asp-validation-for="FullName" class="text-danger"></span>
    </div>
    <div class="mb-3">
        <label asp-for="Phone" class="form-label"></label>
        <input asp-for="Phone" class="form-control" />
        <span asp-validation-for="Phone" class="text-danger"></span>
    </div>
    <div class="mb-3">
        <label asp-for="Email" class="form-label"></label>
        <input asp-for="Email" class="form-control" />
        <span asp-validation-for="Email" class="text-danger"></span>
    </div>
    <div class="mb-3">
        <label asp-for="Category" class="form-label"></label>
        <input asp-for="Category" class="form-control" />
        <span asp-validation-for="Category" class="text-danger"></span>
    </div>
    <div class="mb-3">
        <label asp-for="RatePerUnit" class="form-label"></label>
        <input asp-for="RatePerUnit" class="form-control" />
        <span asp-validation-for="RatePerUnit" class="text-danger"></span>
    </div>

    <button type="submit" class="btn btn-primary">Сохранить</button>
    <a asp-action="Index" class="btn btn-secondary">Отмена</a>
</form>

@section Scripts {
    @await Html.PartialAsync("_ValidationScriptsPartial")
}
```

26. Создайте файл `Edit.cshtml`, скопировав содержимое файла `Create.cshtml` и внеся следующие изменения: заголовок заменяется на «Изменение сведений о переводчике», значение атрибута `asp-action` — на `Edit`, а непосредственно после открывающего тега формы добавляется скрытое поле идентификатора.

```
<input type="hidden" asp-for="Id" />
```

27. Добавьте файл `Delete.cshtml`.

Файл `Views/Translators/Delete.cshtml`:

```
@model TranslationBureau.Application.DTOs.TranslatorDto
@{
    ViewData["Title"] = "Перевод записи в архивное состояние";
}

<h2>Перевод записи в архивное состояние</h2>
```

```

<p>Подтвердите прекращение сотрудничества со следующим переводчиком.</p>

<dl class="row col-md-6">
  <dt class="col-sm-4">ФИО</dt>
  <dd class="col-sm-8">@Model.FullName</dd>
  <dt class="col-sm-4">Телефон</dt>
  <dd class="col-sm-8">@Model.Phone</dd>
  <dt class="col-sm-4">Категория</dt>
  <dd class="col-sm-8">@Model.Category</dd>
  <dt class="col-sm-4">Ставка, руб.</dt>
  <dd class="col-sm-8">@Model.RatePerUnit.ToString("N2")</dd>
</dl>

<form asp-action="Delete" method="post">
  <input type="hidden" name="id" value="@Model.Id" />
  <button type="submit" class="btn btn-danger">Подтвердить</button>
  <a asp-action="Index" class="btn btn-secondary">Отмена</a>
</form>

```

28. Убедитесь в том, что файл `Views/_ViewImports.cshtml` содержит следующие директивы.

```

@using TranslationBureau.Web
@using TranslationBureau.Web.Models
@using TranslationBureau.Application.DTOS
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers

```

Директиву `@using TranslationBureau.Web.Models` удалять не следует: она требуется представлению `Views/Shared/Error.cshtml`, создаваемому шаблоном проекта. Её удаление приводит к ошибке компиляции `CS0246: не удалось найти тип или имя пространства имён «ErrorViewModel»`.

Шаг 13. Создание миграции, запуск и проверка работоспособности

Классы миграций размещаются в проекте слоя инфраструктуры, тогда как строка подключения содержится в файле параметров проекта слоя представления. По этой причине в командах указываются оба проекта.

29. Выполните команду создания миграции.

```

dotnet ef migrations add InitialCreate ^
--project TranslationBureau.Infrastructure ^
--startup-project TranslationBureau.Web

```

30. Примените миграцию к базе данных.

```

dotnet ef database update ^
--project TranslationBureau.Infrastructure ^
--startup-project TranslationBureau.Web

```

31. Запустите приложение.

```

dotnet run --project TranslationBureau.Web

```

В операционных системах семейства Linux и macOS вместо символа `^` для переноса строки применяется символ `\`.

32. Откройте в браузере адрес, указанный в окне терминала, и перейдите по пути `/Translators`.

33. Добавьте сведения о шести переводчиках, приведённые в таблице, что обеспечит формирование двух страниц перечня при размере страницы, равном пяти записям.

ФИО	Телефон	Категория	Ставка, руб.
Иванова Мария	+375291234567	высшая	22.00

Петровна			
Петров Сергей Николаевич	+375331112233	первая	18.50
Сидорова Анна Игоревна	+375445556677	вторая	15.00
Ковалёв Дмитрий Олегович	+375292223344	первая	19.00
Новикова Ольга Вадимовна	+375333334455	высшая	24.50
Лазарев Артём Валерьевич	+375444445566	вторая	14.00

34. Проверьте работу постраничного вывода, перейдя на вторую страницу перечня.
35. Проверьте работу сортировки, щёлкнув по заголовкам столбцов «ФИО» и «Ставка, руб.».
36. Проверьте работу фильтрации, выбрав значение «высшая» в списке категорий.
37. Проверьте срабатывание правил предметной области: попытайтесь сохранить запись с пустым значением поля «ФИО» и с отрицательным значением ставки. Убедитесь в том, что сообщение, сформированное классом `DomainException`, отображается в сводке ошибок формы.
38. Выполните перевод одной из записей в архивное состояние и убедитесь в том, что запись отображается при выборе значения «Архивные» в списке признака сотрудничества.

Проверка соблюдения принципов чистой архитектуры

При приёме результатов работы проверяются следующие положения.

39. **Направление зависимостей.** Проект `TranslationBureau.Domain` не ссылается ни на один другой проект решения и ни на один пакет NuGet. Проект `TranslationBureau.Application` ссылается исключительно на доменное ядро. Проект `TranslationBureau.Infrastructure` ссылается на доменное ядро и слой приложения. Обратные ссылки отсутствуют.
40. **Размещение интерфейсов.** Интерфейс `ITranslatorRepository` объявлен в доменном ядре, его реализация размещена в слое инфраструктуры. Такое размещение соответствует принципу инверсии зависимостей.
41. **Инкапсуляция доменных сущностей.** Свойства сущности `Translator` не содержат открытых методов установки значений. Создание и изменение сущности выполняются исключительно методами `Create` и `Update`, проверяющими правила предметной области.
42. **Размещение правил предметной области.** Проверки размещены в доменной сущности, а не в службе слоя приложения и не в контроллере.
43. **Изоляция технологии доступа к данным.** Единственное упоминание применяемой системы управления базами данных содержится в файле `DependencyInjection.cs` слоя инфраструктуры.
44. **Точка сборки.** Связывание абстракций с конкретными реализациями выполняется в единственном месте — в файле `Program.cs` проекта слоя представления.

Задание для самостоятельного выполнения

Обязательная часть

Обязательная часть задания включает следующие пункты.

45. Создать решение и четыре проекта в соответствии с приведённой структурой, установить ссылки между проектами и убедиться в отсутствии ссылок у проекта доменного ядра.
46. Реализовать доменное ядро: базовый класс `Entity`, класс `DomainException`, сущность `Translator`, обладающую поведением, и интерфейс `ITranslatorRepository`.

47. Реализовать слой приложения: обобщённый класс `PagedResult<T>`, объекты передачи данных, методы преобразования, интерфейс `ITranslatorService` и его реализацию.
48. Реализовать слой инфраструктуры: класс конфигурации сущности, контекст базы данных и репозиторий.
49. Реализовать регистрацию служб посредством методов расширения `AddApplication` и `AddInfrastructure`.
50. Создать контроллер `TranslatorsController` и представления `Index`, `Create`, `Edit` и `Delete`.
51. Создать и применить миграцию, запустить приложение и продемонстрировать работоспособность всех операций, включая страничный вывод, сортировку и фильтрацию.

Дополнительная часть

Дополнительная часть задания предназначена для обучающихся, завершивших обязательную часть ранее отведённого времени.

52. Реализовать доменную сущность `Language` (код языка, наименование, направление письма) и связь «многие ко многим» между переводчиками и языками посредством сущности `TranslatorLanguage`. Дополнить сущность `Translator` методом `SetLanguages`, требующим назначения не менее одного рабочего языка, и методом `KnowsLanguagePair`, проверяющим владение обоими языками указанной пары.
53. Создать проект модульных тестов `TranslationBureau.Domain.Tests`, ссылающийся исключительно на доменное ядро, и реализовать проверки правил предметной области: недопустимости пустого значения ФИО, неположительной ставки оплаты, а также перевода записи в архивное состояние. Убедиться в том, что тесты выполняются без обращения к базе данных.
54. Реализовать интерфейс `IUnitOfWork` в доменном ядре и его реализацию в слое инфраструктуры, перенеся в него метод сохранения изменений.
55. Продемонстрировать замену системы управления базами данных: установить пакет `Microsoft.EntityFrameworkCore.Sqlite`, заменить вызов метода `UseSqlServer` вызовом метода `UseSqlite` и убедиться в том, что изменения затрагивают исключительно слой инфраструктуры, тогда как доменное ядро, слой приложения и слой представления остаются неизменными.
56. Реализовать поиск переводчиков по части фамилии, имени и отчества с сохранением условия поиска при переходе между страницами перечня.

Типичные ошибки и рекомендации преподавателю

Ниже перечислены затруднения, наиболее часто возникающие у обучающихся при выполнении настоящего занятия.

- **Обращение к контексту базы данных непосредственно из контроллера.** Такое решение представляет собой наиболее распространённое нарушение. Следует пояснить, что при указанном подходе замена технологии доступа к данным потребует переработки всех контроллеров, а модульное тестирование окажется невозможным без соединения с базой данных.
- **Размещение интерфейса репозитория в слое инфраструктуры.** Правило формулируется следующим образом: контракт объявляется тем слоем, который его использует, а реализуется тем слоем, который располагает соответствующей технологией. Интерфейс репозитория объявляется в доменном ядре, его реализация размещается в слое инфраструктуры.
- **Ссылка проекта доменного ядра на слой инфраструктуры.** Возникает при попытке применить атрибуты Entity Framework Core к доменным сущностям. Настройка отображения сущностей на таблицы выполняется классами, реализующими интерфейс `IEntityTypeConfiguration<T>`, в слое инфраструктуры.

- **Отсутствие конструктора без параметров у доменной сущности.** Entity Framework Core создаёт экземпляры сущностей при материализации результатов запроса, для чего требуется конструктор без параметров. Такой конструктор объявляется с модификатором `private`, что не препятствует его использованию средствами отражения.
- **Попытка изменить свойство доменной сущности непосредственным присваиванием.** Компилятор сообщает об ошибке, поскольку методы установки значений объявлены закрытыми. Следует пояснить, что указанное ограничение является намеренным: изменение состояния сущности допускается исключительно посредством её методов, что гарантирует соблюдение правил предметной области.
- **Отсутствие параметра `--startup-project` в командах миграции.** При отсутствии указанного параметра средство `dotnet-ef` не обнаруживает файл параметров приложения и завершается сообщением о невозможности создания экземпляра контекста базы данных.
- **Сомнение в допустимости установки пакета `Microsoft.EntityFrameworkCore` в слой приложения.** Следует пояснить, что указанный пакет содержит исключительно абстракции и методы расширения для `IQueryable<T>`; провайдер конкретной системы управления базами данных в слой приложения не устанавливается, вследствие чего независимость слоя сохраняется.